

Secure File Vault

Complete Objective Project Report

Report purpose: Document the current implementation, architecture, security model, workflows, data structures, operational requirements, and known limitations of the Secure File Vault project.

Review basis: Source code, SQL schema, templates directory, configuration examples, requirements, and project guides present in the workspace on 2026-09-07. This is an objective implementation report, not a claim of production certification.

Attribute	Value
Application type	Authenticated Flask web application for encrypted file storage
Primary language	Python 3.10+
Data store	SQLite (vault.db)
Encryption	AES-256-GCM with per-file data keys wrapped by a master key
Web interface	Flask/Jinja templates with static CSS
Current status	Functional learning/project implementation; production hardening remains

1. Executive Summary

Secure File Vault is a small web application that allows an authenticated user to register, log in, upload files, store only encrypted content on disk, decrypt and download owned files, delete files, and create expiring one-time share links. The application also records security-relevant actions in an audit log and exposes integrity and risk information through the user interface.

The implementation combines Flask, Flask-WTF, SQLite, Werkzeug password hashing, python-dotenv, and the cryptography package. The main protection boundary is enforced by authenticated routes and owner-scoped database queries. Files are encrypted before being written to the `encrypted_files` directory. Passwords are stored as slow salted password hashes rather than plaintext or fast general-purpose hashes.

The project has a clear educational security focus. It includes useful defensive controls such as CSRF protection, security headers, session expiry, account lockout, generic login errors, random storage names, authenticated encryption, file fingerprints, a hash-chained audit trail, conservative upload screening, and local URL risk analysis. It should still be treated as a learning system until HTTPS deployment, production serving, operational key management, stronger rate limiting, testing, and monitoring are added.

2. Project Objectives and Scope

Primary objective: Provide a simple, self-contained vault in which users can store files while protecting confidentiality and detecting unauthorized modification.

- Authenticate users with hashed passwords and session-based access control.
- Encrypt file content before persistence using authenticated encryption.
- Ensure a user can access only their own file records through normal vault routes.
- Record authentication, file, sharing, integrity, and URL analysis events.
- Detect suspicious uploads and provide a local URL safety assessment.
- Support operational visibility through dashboards, audit views, risk scoring, and CSV export.

In scope: application source, SQLite schema, local encrypted-file storage, browser UI, configuration, and security mechanisms implemented in the repository. **Out of scope:** cloud deployment, external identity providers, malware scanning, network URL reputation lookups, formal penetration testing, and production key custody.

3. Technology Stack

Component	Role
Flask	HTTP application, routing, sessions, request lifecycle, templates
Flask-WTF	CSRF protection for submitted forms
cryptography	AES-GCM encryption, decryption, and key generation
Werkzeug	Password hash generation and verification; safe filenames
python-dotenv	Loads local secrets from <code>.env</code>
SQLite	Users, file metadata, audit events, and share-link records
Jinja2 / CSS	Browser-rendered pages and visual styling

4. Architecture and Component Responsibilities

The application uses an application factory and Flask Blueprints. `create_app()` loads configuration, validates required secrets, creates the encrypted storage directory, initializes CSRF protection and the database, registers

authentication, vault, and URL Sentinel routes, and adds security response headers.

Module or directory	Responsibility
app.py	Application factory, extension setup, blueprint registration, security headers, local entry point
config.py	Paths, key identifiers, upload limit, session settings, and lockout policy
generate_keys.py	Generates random base64-encoded SECRET_KEY and MASTER_KEY values
db.py	Request-scoped SQLite connection, schema initialization, migrations, and legacy backfills
schema.sql	Definitions for users, files, audit_log, and share_links tables
auth.py	Registration, login, failed-attempt tracking, lockout, logout
decorators.py	login_required and idle-session enforcement
crypto_utils.py	AES-256-GCM primitives and version-2 envelope encryption
vault.py	Dashboard, profile, upload, download, delete, sharing, integrity, audit routes
audit.py	Audit event creation and hash-chain verification
risk.py	Explainable, time-decaying event risk score
threat_scan.py	Extension, magic-header, macro, and empty-payload screening
url_scanner.py / url_sentinel.py	Local URL indicator analysis and authenticated UI route
templates/ and static/	HTML pages and CSS presentation
encrypted_files/	Randomly named ciphertext blobs; not original filenames

5. Core User Workflows

5.1 Registration and authentication

A user submits a username, password, and confirmation. The implementation requires a username of at least three characters and a password of at least eight characters. The password is processed by Werkzeug's default password hashing function, documented in the project as script-based, and only the resulting hash is inserted into users.

During login, the account lockout state is checked first. A valid login clears failed attempts, creates a session containing the user ID and username, enables Flask's permanent session behavior, and stores a UTC last-active timestamp. Invalid credentials produce the same generic error for unknown usernames and wrong passwords, reducing account enumeration. Five failures for an existing account trigger a 15-minute lockout.

5.2 Upload and encryption

The upload route requires an authenticated session. It normalizes the submitted filename with `secure_filename()`, reads the payload, scans it, calculates a SHA-256 plaintext fingerprint, encrypts it with envelope encryption, and writes only ciphertext to a randomly generated UUID-based `.enc` filename. Metadata, nonces, wrapped key material, key ID, fingerprint, and size are stored in the files table.

5.3 Download and integrity protection

The download route selects a file by both file ID and the current session user ID. This owner condition is the direct authorization boundary. The ciphertext is decrypted and authenticated before being returned as a download. An `InvalidTag` exception is treated as an integrity failure and is recorded in the audit trail instead of returning data.

5.4 Sharing

An owner can create a share link with an expiry choice of 1, 24, or 168 hours. The raw token is returned in the generated link, while only its SHA-256 hash is stored. A share is limited by expiration and `max_downloads`, which defaults to one. Redemption increments the count under a database condition before decrypting the file.

5.5 Monitoring and analysis

The dashboard provides file search, file totals, audit-event counts, failed-event counts, a bounded security score, recent activity, and calculated risk. The profile page shows account, file, share, IP, key ID, and recent event information. The audit page displays recent entries, severity summaries, chain status, and CSV export. Authenticated API routes perform owned-file integrity scans and audit-chain checks.

6. Cryptography and Key Management

The current upload path uses version-2 envelope encryption. A fresh random 256-bit data key is generated for each file. The plaintext is encrypted with AES-GCM using a random 12-byte content nonce and the configured key ID as associated data. The data key is then encrypted with the 256-bit `MASTER_KEY` using a separate random 12-byte wrapping nonce and the same key ID as associated data. The database stores the ciphertext metadata, content nonce, wrapped data key, wrapping nonce, and key ID.

AES-GCM supplies confidentiality and authenticity. Decryption fails when the ciphertext, nonce, associated key ID, or key is incorrect. The separate SHA-256 fingerprint gives an additional plaintext identity check during integrity scans. Legacy version-1 rows are still supported through `decrypt_bytes()`, and `db.py` contains migration/backfill logic for older records.

Secret or value	Storage / purpose
<code>SECRET_KEY</code>	Environment variable; signs Flask session data
<code>MASTER_KEY</code>	Environment variable; base64-encoded 32-byte key that wraps per-file data keys
<code>KEY_ID</code>	Configuration label included as encryption associated data and stored with file metadata

Secret or value	Storage / purpose
Data key	Random per-file AES key, wrapped by MASTER_KEY and stored only in encrypted form
Share token	Raw token appears in the link; SHA-256 token hash is stored in SQLite

A lost MASTER_KEY makes the encrypted files unrecoverable. Database and encrypted_files backups must remain paired. The code does not provide an external key vault, rotation workflow, or key escrow process.

7. Database Design

Table	Important fields and purpose
users	id, username, password_hash, failed_attempts, lockout_until, created_at
files	Ownership, original and random stored names, nonce, encryption version, key ID, wrapped key, wrap nonce, SHA-256, size, timestamp
audit_log	User context, action, filename, IP, timestamp, details, previous hash, entry hash
share_links	File/user ownership, token hash, expiry, max downloads, current downloads, creation time

SQLite foreign keys are enabled on request-scoped connections. The initialization process creates missing tables, adds newer columns to older databases, backfills audit hashes in insertion order, and backfills file fingerprints for legacy encrypted rows where the file is available.

8. Security Controls

Control	Implementation and purpose
Password protection	Werkzeug salted password hashes; no plaintext password storage
CSRF defense	Flask-WTF CSRFProtect initialized for form submissions
Session protection	HTTPOnly, SameSite=Lax, optional Secure flag, permanent lifetime, idle timeout
Authorization	login_required decorator and user_id predicates on owned-file queries
Login abuse control	Per-account failed-attempt counter and 15-minute lockout after five failures
Enumeration resistance	Generic invalid-credential response for missing users and wrong passwords
Path safety	secure_filename plus random UUID storage names
Transport/browser headers	nosniff, frame denial, strict referrer policy, permissions policy, conditional HSTS
Tamper detection	AES-GCM authentication, SHA-256 fingerprints, and chained audit entry hashes
Upload screening	Blocks selected executable types and suspicious magic headers; flags macro files and empty files
URL screening	Local analysis of schemes, IP hosts, punycode, shorteners, encoding, terms, ports, and density

9. Routes and Browser Features

Route	Method	Access	Function
/register	GET/POST	Public	Create account
/login	GET/POST	Public	Authenticate and enforce logout
/logout	GET	Session	Clear session and record logout
/	GET	Authenticated	Dashboard, search, statistics, recent events
/profile	GET	Authenticated	Account and security profile
/upload	POST	Authenticated	Scan, encrypt, store, and record file
/download/	GET	Authenticated	Decrypt and download owned file
/delete/	POST	Authenticated	Remove owned encrypted file and metadata
/share/	POST	Authenticated	Create expiring one-use share
/shared/	GET	Token	Redeem a valid share link
/api/integrity-scan	POST	Authenticated	Verify owned blobs without returning plaintext
/api/audit-integrity	POST	Authenticated	Verify the global audit hash chain
/audit-log	GET	Authenticated	View recent audit events and integrity state
/audit-log.csv	GET	Authenticated	Export up to 2,000 audit entries
/url-sentinel	GET/POST	Authenticated	Analyze URL indicators locally

10. Threat Model and Risk Interpretation

The design primarily addresses an attacker who can read ordinary stored files, guess file IDs, submit hostile form input, tamper with ciphertext, or attempt password guessing against an account. It does this through encryption, authenticated decryption, owner-scoped queries, CSRF protection, input normalization, generic errors, logout, and audit visibility.

The risk score is an explainable indicator based on recent audit events. Weighted events include failed login, logout, failed integrity verification, shared download, and deletion. Contributions decay by age over a 24-hour half-life and are bounded at 100. The score describes observable account events; it is not a full host, malware, or network security assessment.

The URL Sentinel is intentionally offline. It makes no network request and cannot confirm reputation, redirects, certificate validity, domain ownership, or current threat intelligence. The upload Threat Gate is likewise a conservative heuristic and is explicitly not antivirus.

11. Installation and Operation

Required environment: Python 3.10 or newer and a virtual environment. Install dependencies from requirements.txt, run generate_keys.py, place the generated SECRET_KEY and MASTER_KEY in a local .env file, and start the application with python app.py. The default development address is http://127.0.0.1:5000.

```
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
python generate_keys.py
python app.py
```

The application creates the database schema and encrypted_files directory as needed. The configured upload limit is 50 MB. Real deployment requires a WSGI server, HTTPS, SESSION_COOKIE_SECURE=true, protected secrets,

controlled backups, and production logging.

12. Verification and Test Guidance

The project guides state that the following flows were exercised during development: registration, login, logout, encrypt/decrypt round-trip, tamper detection, cross-user access control, and idle-session expiry. The visible workspace does not contain a dedicated automated test suite, so these claims should be reproduced in the current environment before release.

Test area	Expected verification
Registration	Short usernames/passwords and mismatched confirmation are rejected; valid account is stored with a hash
Authentication	Valid login succeeds; unknown user and wrong password have the same visible error
Logout	Five failed attempts for an existing account block login for 15 minutes
Encryption	Stored blob is not plaintext; download matches original bytes
Tamper detection	Changing the encrypted blob produces integrity failure and no plaintext download
Authorization	A user cannot download, delete, or share another user's file ID
Sharing	A link expires and cannot be redeemed after its one allowed download
Audit chain	Normal events verify; editing an audit row makes the chain invalid
Integrity API	Owned files report VERIFIED, mismatch, or failure accurately
URL Sentinel	Dangerous schemes and high-risk indicators produce the expected verdict

13. Limitations and Recommended Improvements

- Deploy behind HTTPS and enable the Secure session-cookie flag.
- Use a production WSGI server and disable debug mode outside local development.
- Add IP-based and preferably distributed rate limiting to complement per-account logout.
- Protect MASTER_KEY with a dedicated secrets manager or key-management service and define rotation procedures.
- Add automated unit, integration, security-regression, and migration tests.
- Consider database/file transaction recovery for failures between writing a blob and committing metadata.
- Add operational cleanup for expired share-link rows and orphaned encrypted blobs.
- Define backup, restore, retention, audit export, and incident-response procedures.
- Consider malware scanning and content-type policy appropriate to the deployment.
- Review concurrency behavior for simultaneous share redemption and multi-process SQLite operation.
- Add security monitoring and alerting rather than relying only on a user-facing risk score.
- Review whether the global audit chain and user-visible event scope match the desired multi-user trust model.

14. Conclusion

Secure File Vault is a coherent educational secure-storage application with a focused architecture and a broad set of defensive features. Its strongest properties are authenticated encryption, per-file key wrapping, owner-scoped access checks, password hashing, session controls, audit-chain verification, and explicit integrity handling. The project is suitable for learning and controlled local demonstrations. It is not yet a production-ready file service without the

deployment, key-management, testing, rate-limiting, backup, and operational improvements listed above.

End of report.